Cloud Computing Lab 08

Submitted To:

Sir Shoaib & Sir Waqas

Submitted By:

Hamail Fatima

Roll No:

2023-BSE-023

Section:

5(A)

Task 1 — Create an AWS account and enable UAE (me-central-1)

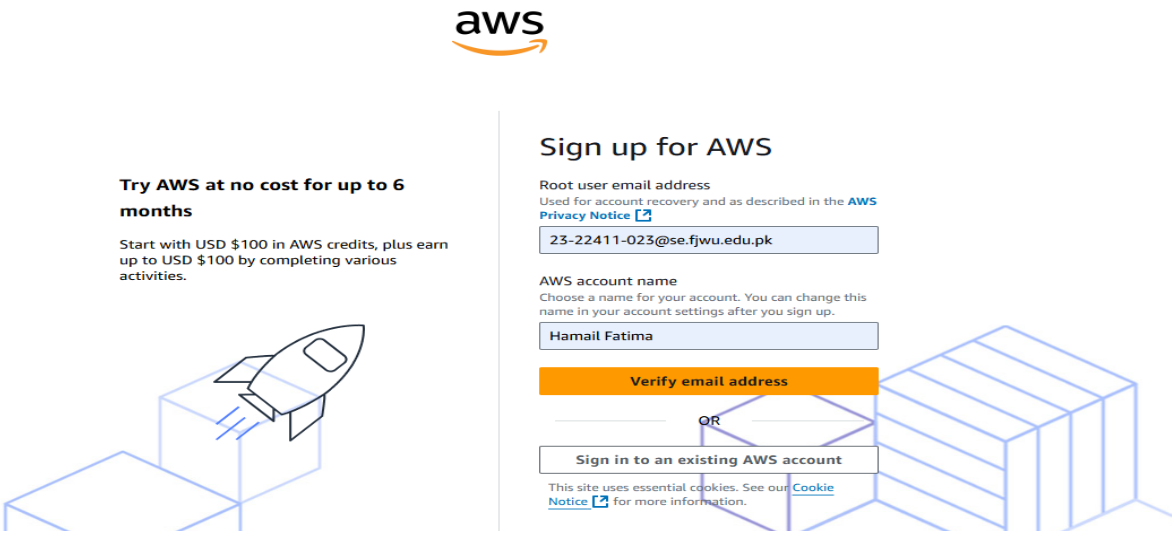
Goal: Register a new AWS root account and enable the UAE me-central-1 region.

Do these steps in order and capture a screenshot immediately after each numbered step. Place screenshots in Lab8/screenshots/.

Steps and required screenshots:

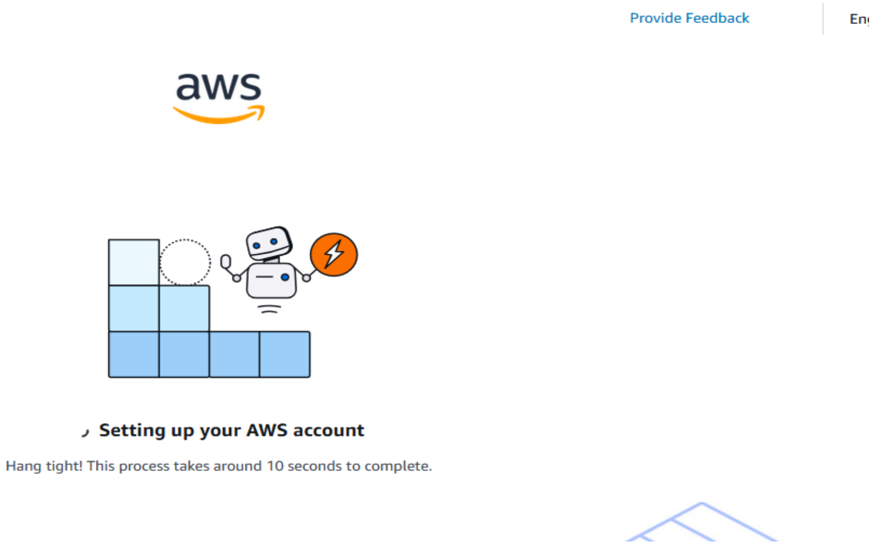
1. Open your browser and go to: [AWS Signup]

Save screenshot as: `task1_open_signup_page.png` — browser showing the signup page.



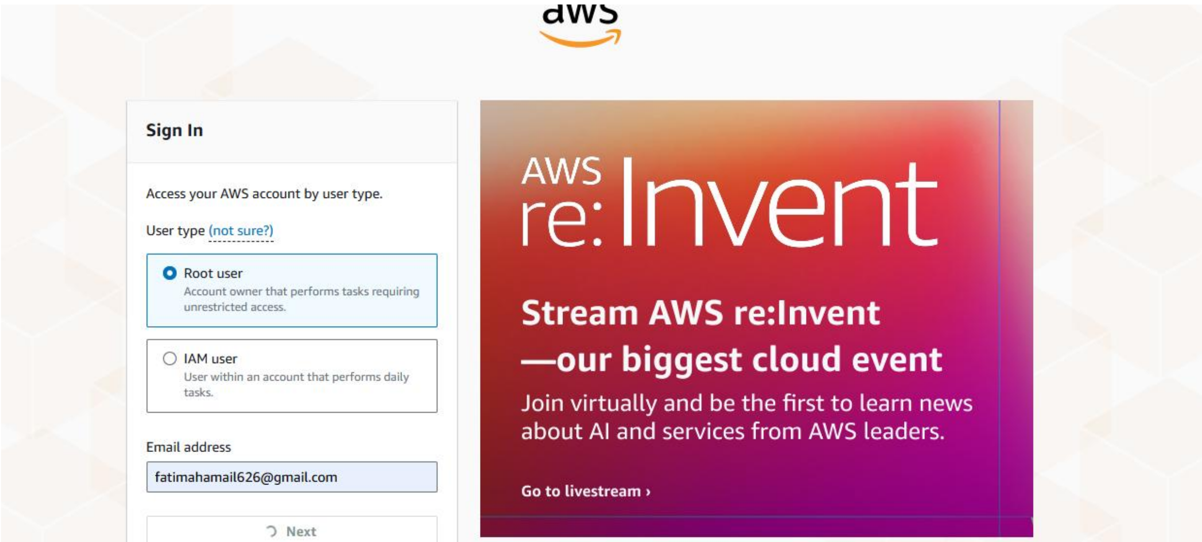
2.Complete registration (Account type: Personal, Plan: AWS Paid Plan), fill contact, billing (credit card) and phone details, complete verification. After successful registration capture:

Save screenshot as: task1_signed_up_confirmation.png — registration success/confirmation page or payment confirmation (do NOT include credit card full details).



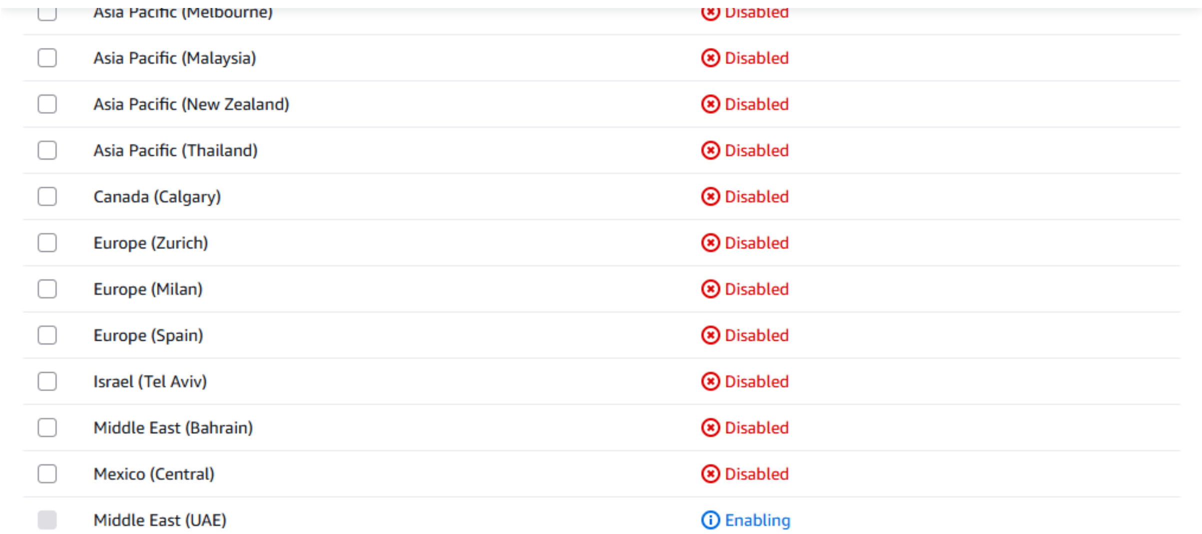
3.Sign in as the root user (root email). Immediately capture:

Save screenshot as: task1_root_signed_in.png — AWS Console Home after root login (top bar with root email/account alias visible).



4.From the Console, open the region selector and enable UAE (me-central-1), then switch to me-central-1. Capture the change:

- Save screenshot as: task1_enable_region_me-central-1.png — region selector showing me-central-1 selected.



5.Task 1 summary screenshot (combine evidence):

- Save screenshot as: task1_summary.png — single screenshot showing root console header (root email/account alias) and region set to me-central-1.

Task 2 — Create IAM Admin and Lab8User with console access

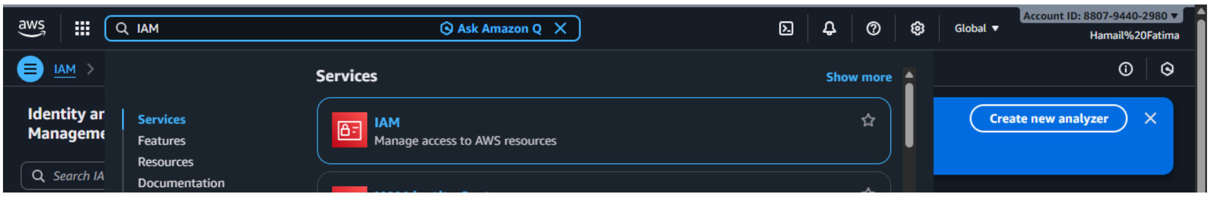
Goal: Create an IAM user named Admin (console access + AdministratorAccess policy), download the .csv, logout root, then login as the IAM Admin and create Lab8User with same permissions.

Do these steps in order and capture a screenshot immediately after each numbered step/substep. Place screenshots in Lab8/screenshots/.

Steps and required screenshots:

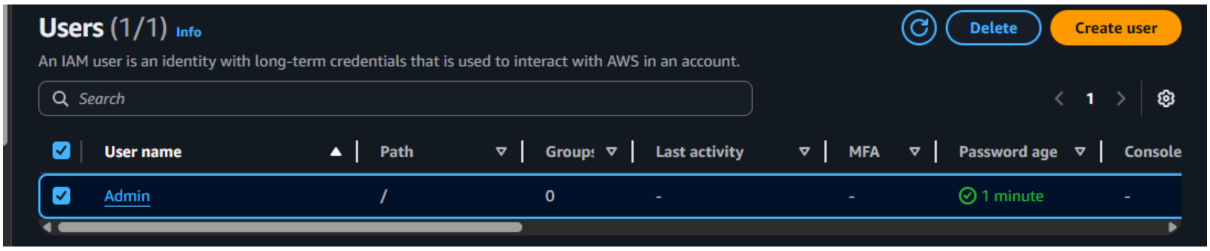
1. Open IAM via Console search (Alt+S → "IAM").

   Save screenshot as: `task2_open_iam_console.png` — IAM console landing page (region me-central-1 visible).



2.Capture the completion screen when user is created:

- Save screenshot as: task2_admin_create_confirmation.png — IAM "Create user" success screen showing Admin (do NOT include password).



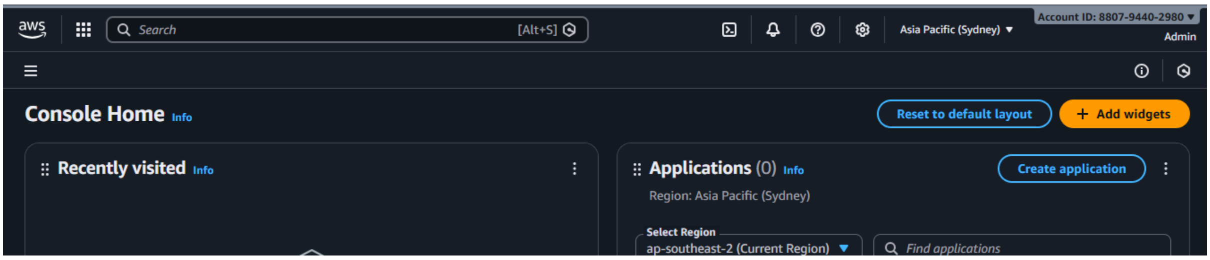
3. Download the Admin .csv and show its presence on your Windows host (do not display the password text):

- Save screenshot as: task2_admin_csv_and_signin_url.png — Windows File Explorer showing the downloaded CSV filename and/or a cropped view of the CSV showing only the Sign-in URL and username (redact the password if visible).

A	B	C	D	E	F	G
User name	Password	Console sign-in URL				

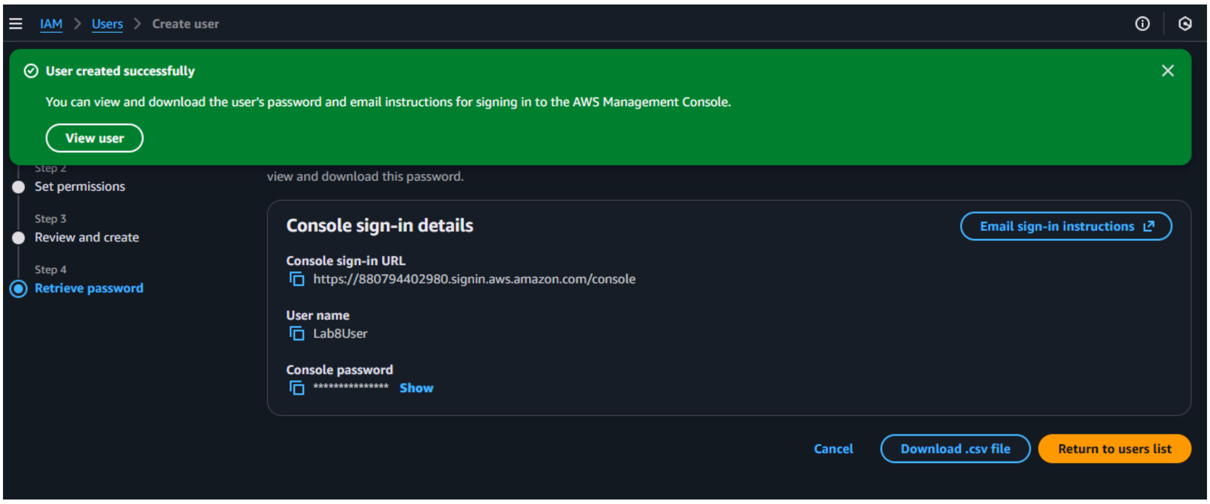
4. Sign out of root, then sign in using the Admin account (use the signin URL from the .csv). Capture after successful Admin login:

- Save screenshot as: task2_admin_console_after_login.png — Admin user console home.



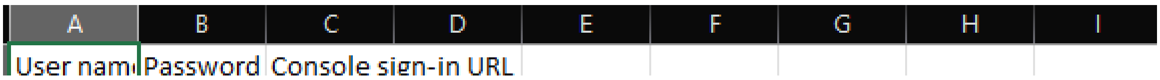
5.While logged in as Admin, create Lab8User:

- IAM → Users → Create user
- Username: Lab8User
- Provide user access to the AWS Management Console
- Attach AdministratorAccess policy
  Capture the create-user success screen:
- Save screenshot as: task2_create_lab8user_and_csv.png — Lab8User create confirmation and CSV download prompt (do NOT include password)



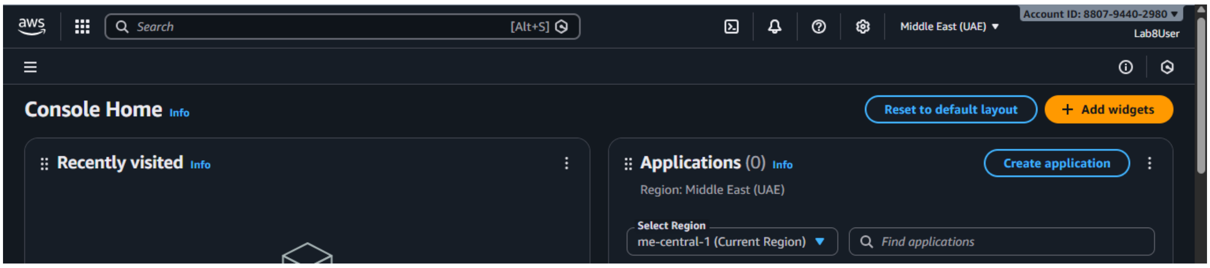
6.Download/save the Lab8User CSV on your Windows host (do not show password).

- Save screenshot as: task2_lab8user_csv_saved.png — File Explorer showing the Lab8User CSV filename (cropped to exclude sensitive content).



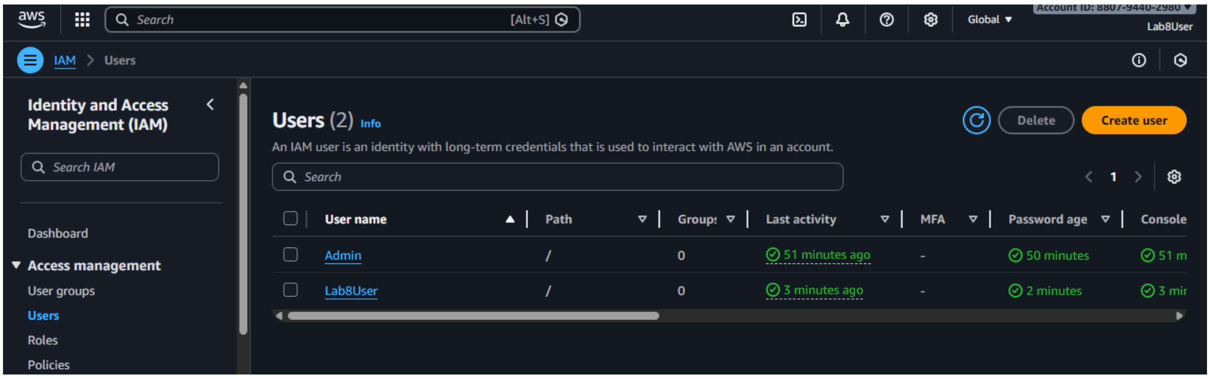
7.Logout Admin and login as Lab8User (use the Lab8User signin URL and credentials). Capture after login:

- Save screenshot as: task2_lab8user_logged_in.png — Lab8User console home.



8. Task 2 summary (combine evidence):

- Save screenshot as: task2_summary.png — IAM Users list showing both Admin and Lab8User present (region me-central-1 visible).



Task 3 — Inspect VPC resources (in UAE me-central-1)

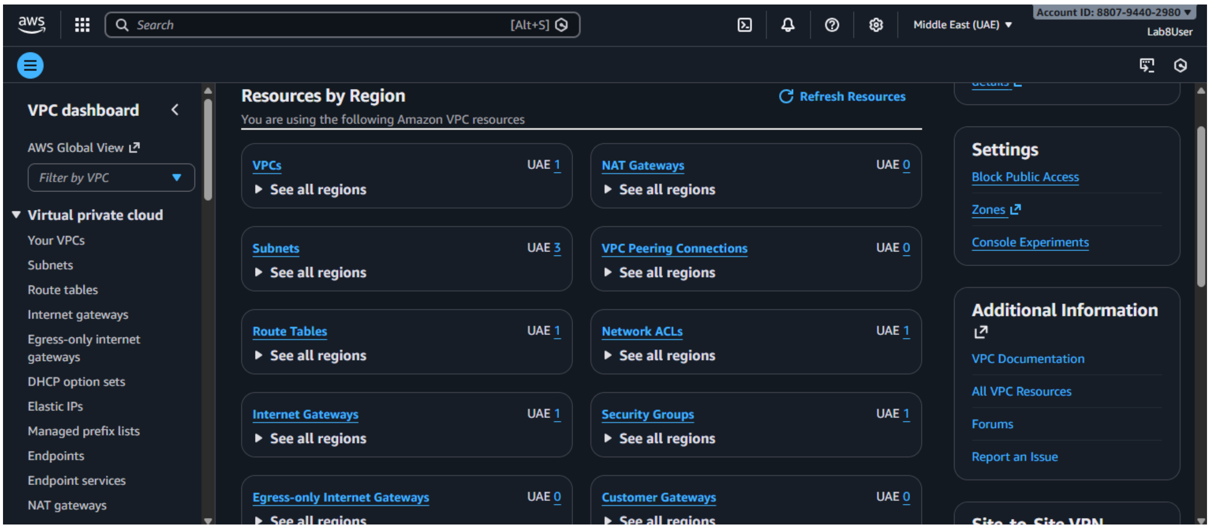
Goal: In the VPC console list the counts of specific resources.

Do these steps in order and capture a screenshot immediately after viewing each page. Place screenshots in Lab8/screenshots/.

Steps and required screenshots:

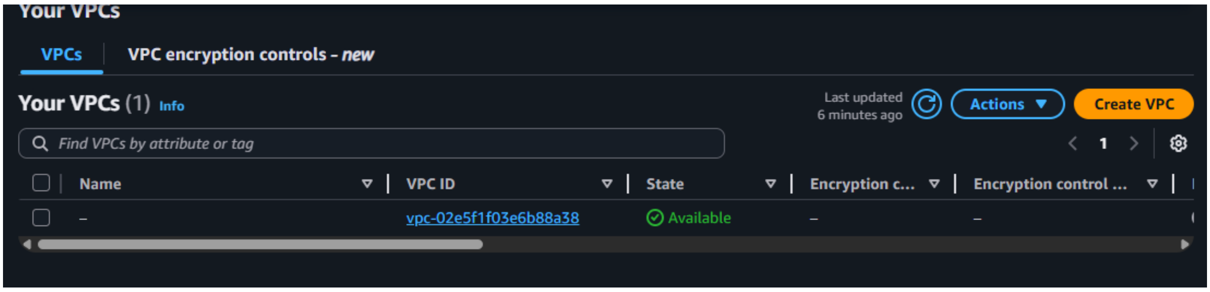
1. Open VPC console (Alt+S → "VPC") while region is me-central-1.

   Save screenshot as: `task3_open_vpc_console.png` — VPC console landing page (region visible).



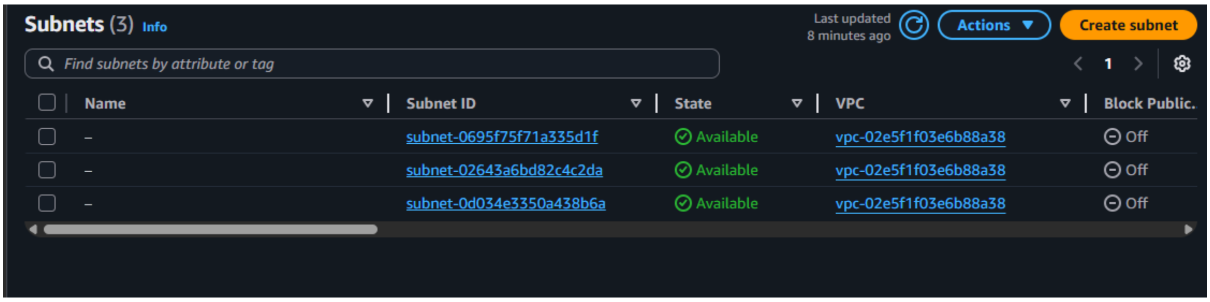
2.View VPCs list. Capture:

- Save screenshot as: task3_vpcs_list.png — VPCs list view (show default VPC if present).



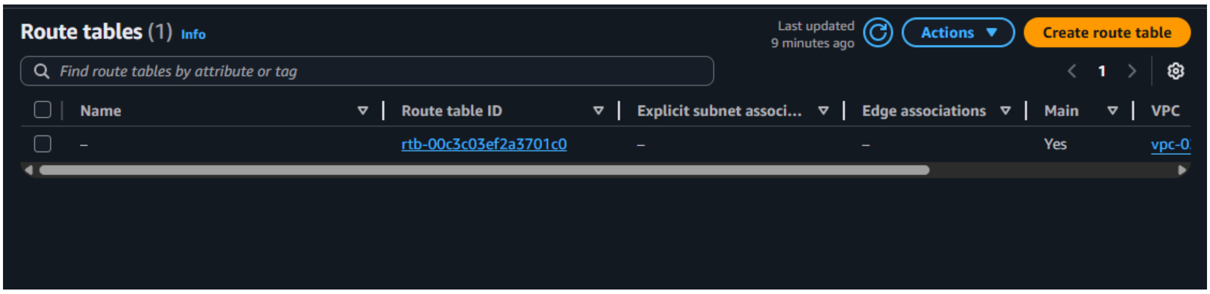
3. View Subnets list. Capture:

- Save screenshot as: task3_subnets_list.png — Subnets list view (show at least 3 default subnets if present).



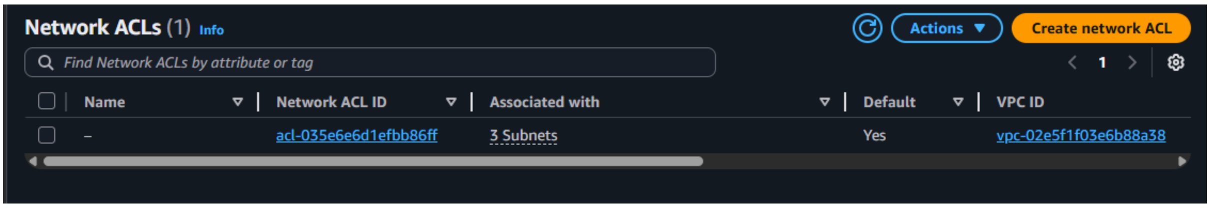
4.View Route Tables list. Capture:

- Save screenshot as: task3_route_tables_list.png — Route Tables list view.



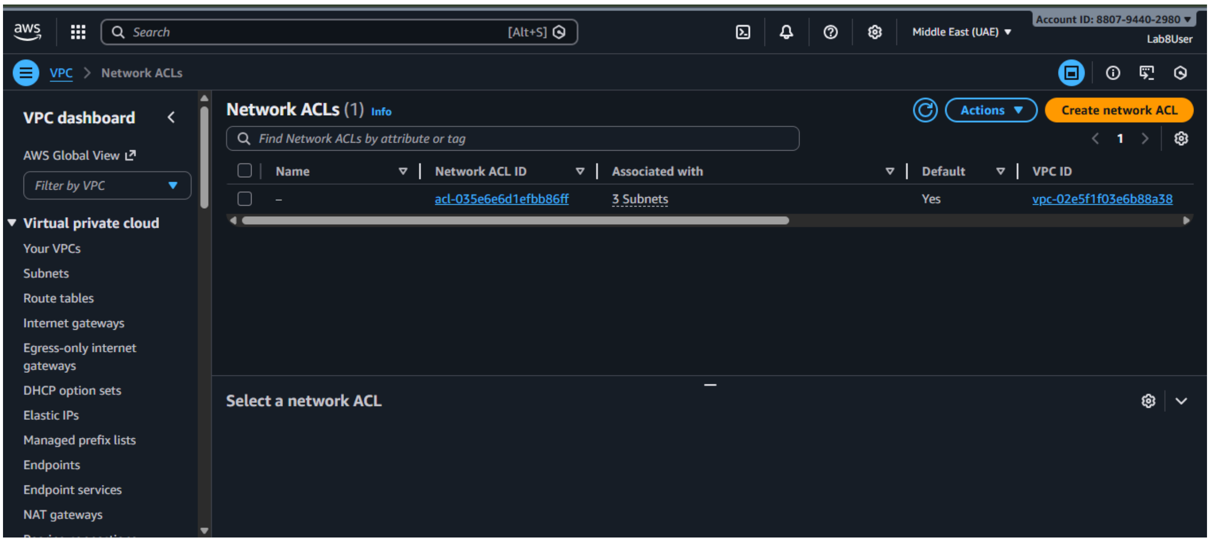
5.View Network ACLs list. Capture:

- Save screenshot as: task3_network_acls_list.png — Network ACLs list view.



6.Task 3 summary (combine evidence):

- Save screenshot as: task3_summary.png — a single screenshot showing the VPC console left navigation and counts or multiple open tabs/windows tiled to show each resource's list (region me-central-1 visible).



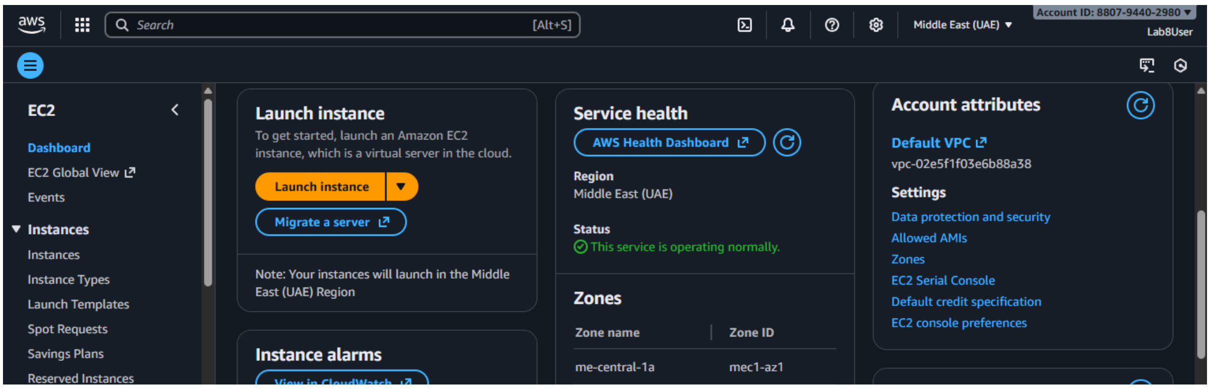
Task 4 — Launch EC2, SSH, install Docker & Docker Compose, deploy Gitea

Goal: Launch an EC2 instance named Lab8Machine, configure security group Lab8SecurityGroup allowing SSH from your IP, create an ED25519 key pair Lab8Key (.pem), SSH from Windows host, install Docker & Docker Compose, create compose.yaml from the repository, run docker compose up -d, allow inbound TCP port 3000, and open Gitea UI.

Steps and required screenshots:

1. Open EC2 Console (Alt+S → "EC2") (me-central-1).

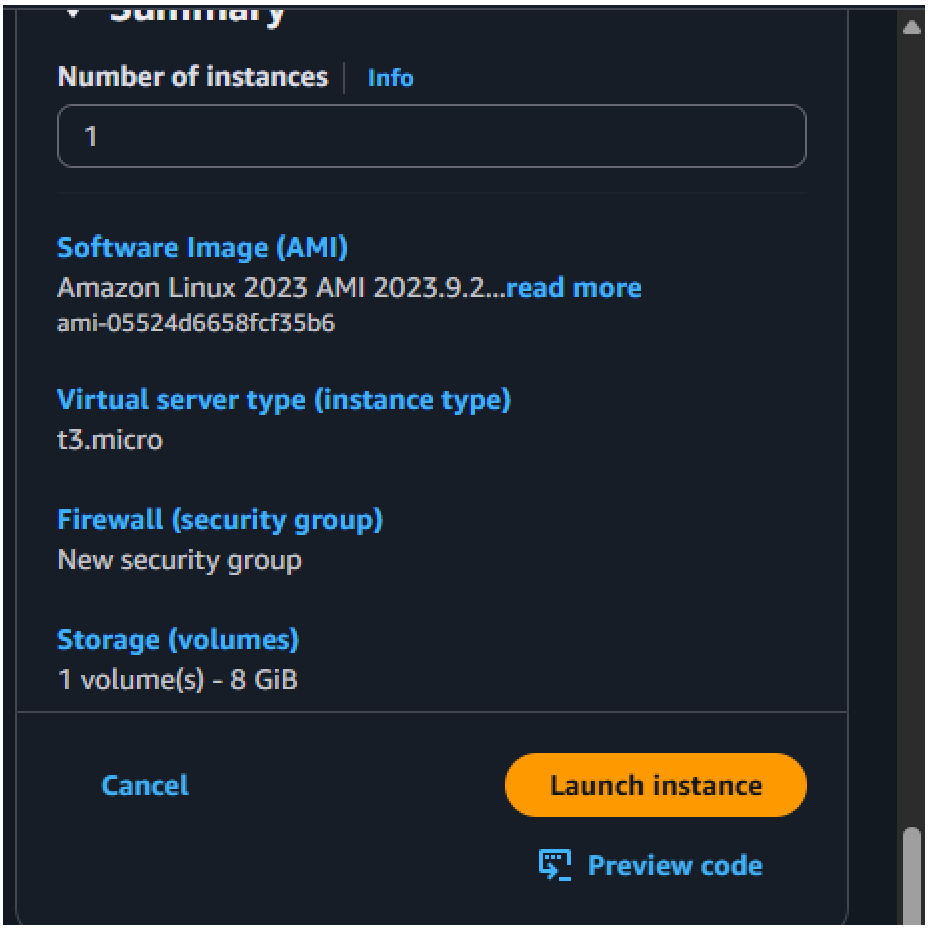
   Save screenshot as: task4_open_ec2_console.png — EC2 console landing page with region visible



Instance Launch configuration (during review before launching). Configure:

- Name: Lab8Machine

- Save screenshot as: `task4_launch_instance_config.png` — final review page showing instance name, AMI, type, security group, key pair.
- Save screenshot as: `task4_keypair_download.png` — Windows File Explorer showing Lab8Key.pem downloaded (do NOT open .pem contents).



2.After launch, EC2 Instances list showing Lab8Machine in "running" state and public IPv4 visible

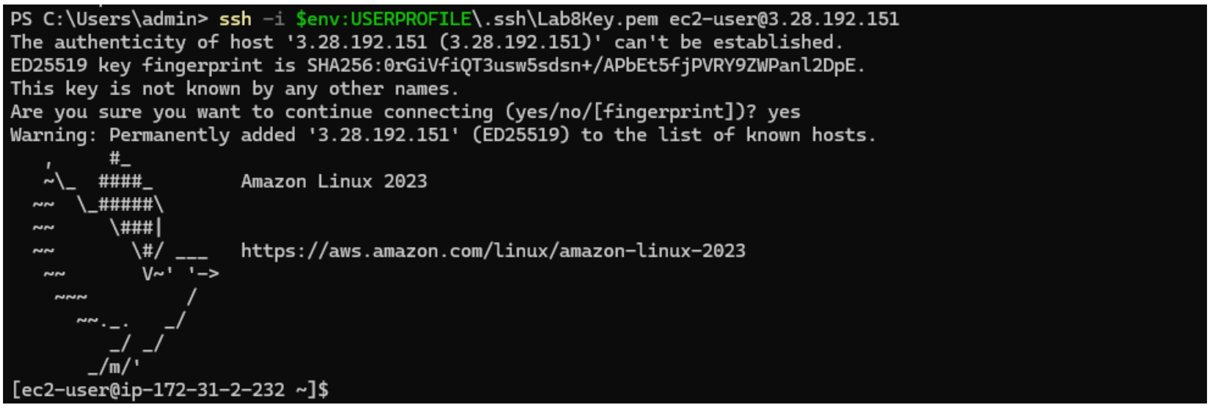
- Save screenshot as: task4_instance_running_console.png — Instances table with Lab8Machine running and Public IPv4.

4.On Windows host, run SSH using the downloaded .pem (PowerShell/Git Bash/Windows Terminal):

ssh -i <path>/Lab8Key.pem ec2-user@<public-IP>

Capture the SSH command and successful shell prompt on the EC2 instance:

- Save screenshot as: task4_ssh_from_windows_to_ec2.png — PowerShell showing ssh command and EC2 shell (do NOT show private key contents).



Run the install commands on the EC2 shell:

sudo yum update -y

sudo yum install -y docker

sudo mkdir -p /usr/local/lib/docker/cli-plugins

sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64 -o /usr/local/lib/docker/cli-plugins/docker-compose

sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose

sudo systemctl start docker

Capture the terminal showing these commands run and successful outputs:

- Save screenshot as: task4_ec2_install_docker_compose_started.png — outputs of update/install and systemctl start.

```
[ec2-user@ip-172-31-2-232 ~]$ sudo yum update -y
sudo yum install -y docker
sudo mkdir -p /usr/local/lib/docker/cli-plugins
sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64 -o /usr/local/lib/d
ocker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
sudo systemctl start docker
Amazon Linux 2023 Kernel Livepatch repository                                    240 kB/s |  29 kB     00:00
Dependencies resolved.
Nothing to do.
Complete!
Last metadata expiration check: 0:00:02 ago on Mon Dec 15 15:03:05 2025.
Dependencies resolved.
========================================================================================================================
 Package                            Architecture        Version                            Repository              Size
========================================================================================================================
Installing:
 docker                             x86_64              25.0.13-1.amzn2023.0.2             amazonlinux             46 M
Installing dependencies:
 container-selinux                  noarch              4:2.242.0-1.amzn2023               amazonlinux             58 k
 containerd                         x86_64              2.1.5-1.amzn2023.0.1               amazonlinux             23 M
 iptables-libs                      x86_64              1.8.8-3.amzn2023.0.2               amazonlinux            401 k
 iptables-nft                       x86_64              1.8.8-3.amzn2023.0.2               amazonlinux            183 k
```

6.Create/edit compose.yaml on the EC2 instance (sudo vim compose.yaml) and paste content from the repo: Gitea . While pasting, capture the editor content:

- Save screenshot as: task4_vim_compose_yaml_paste.png — vim editor showing compose.yaml contents while pasted.

```yaml
services:
  gitea:
    image: gitea/gitea:latest
    container_name: gitea
    environment:
      - DB_TYPE=postgres
      - DB_HOST=db:5432
      - DB_NAME=gitea
      - DB_USER=gitea
      - DB_PASSWD=gitea
    restart: always
    volumes:
      - gitea:/data
    ports:
      - 3000:3000
    extra_hosts:
      - "www.jenkins.com:host-gateway"
    networks:
      - webnet
  db:
    image: postgres:alpine
    container_name: gitea_db
    environment:
      - POSTGRES_USER=gitea
      - POSTGRES_PASSWORD=gitea
      - POSTGRES_DB=gitea
    restart: always
    volumes:
      - gitea_postgres:/var/lib/postgresql/data
```

7.Save and verify file exists:

- Save screenshot as: task4_compose_yaml_saved_ls.png — ls -l showing compose.yaml present.

```
[ec2-user@ip-172-31-2-232 ~]$ cd ~
[ec2-user@ip-172-31-2-232 ~]$ sudo vim compose.yaml
[ec2-user@ip-172-31-2-232 ~]$ ls -l compose.yaml
-rw-r--r--. 1 root root 1126 Dec 15 15:09 compose.yaml
[ec2-user@ip-172-31-2-232 ~]$
```

8.Add ec2-user to docker group, show groups before re-login, exit and reconnect, show groups after reconnect:

groups   # user does not docker permission

sudo usermod -aG docker $USER

groups   # before re-login

exit

Reconnect

ssh -i <path>/Lab8Key.pem ec2-user@<public-IP>

groups   # after re-login (should include docker)

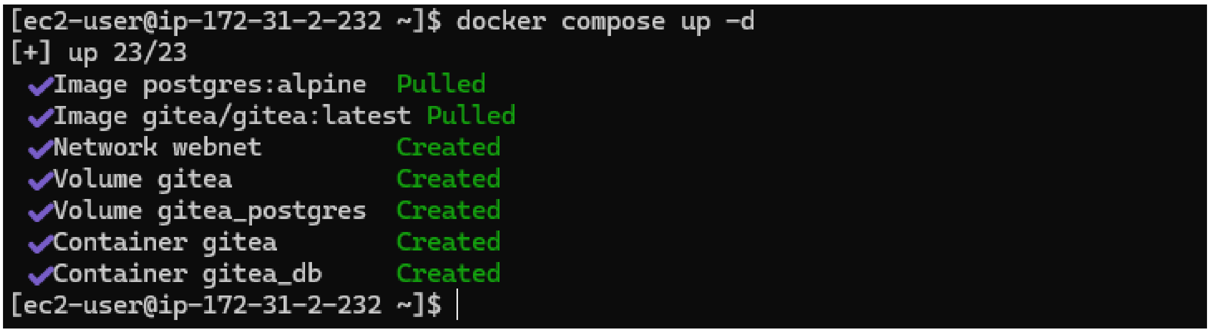
- Save screenshot as: task4_usermod_and_groups_before_after.png — show usermod command, groups output before exit, reconnect sequence, and groups output after (docker included).

```
[ec2-user@ip-172-31-2-232 ~]$ groups
ec2-user adm wheel systemd-journal
[ec2-user@ip-172-31-2-232 ~]$ sudo usermod -aG docker $USER
[ec2-user@ip-172-31-2-232 ~]$ groups
ec2-user adm wheel systemd-journal
[ec2-user@ip-172-31-2-232 ~]$ exit
logout
Connection to 3.28.192.151 closed.
PS C:\Users\admin> ssh -i Lab8Key.pem ec2-user@3.28.192.151
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Mon Dec 15 15:00:30 2025 from 103.149.240.116
[ec2-user@ip-172-31-2-232 ~]$ groups
ec2-user adm wheel systemd-journal docker
[ec2-user@ip-172-31-2-232 ~]$
```

10.Run docker compose up -d from the directory with compose.yaml:

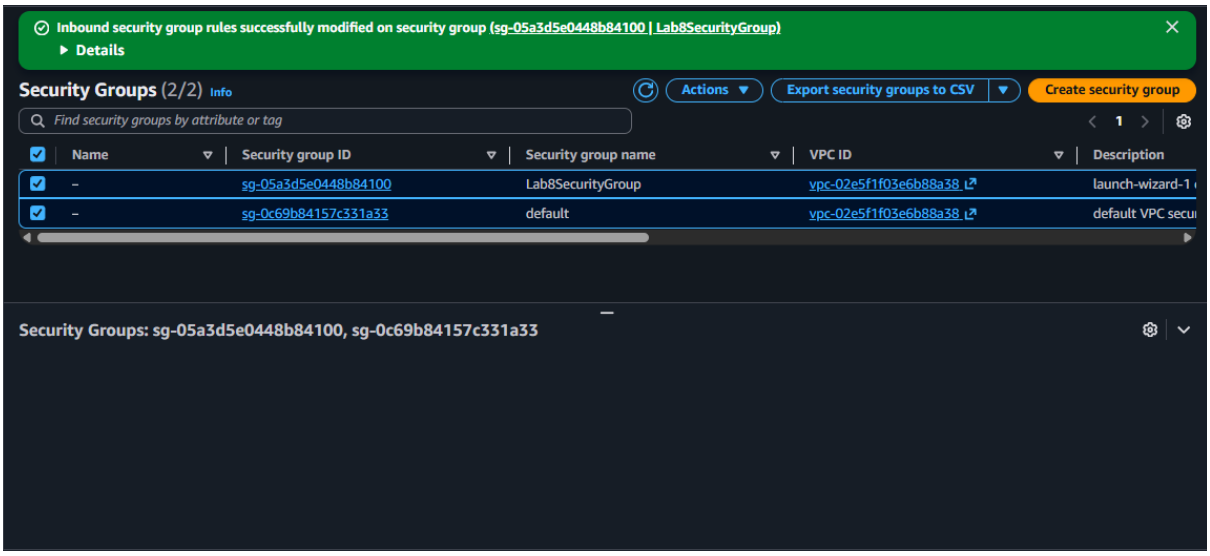
docker compose up -d

- Save screenshot as: task4_docker_compose_up.png — output of docker compose up -d showing containers starting.



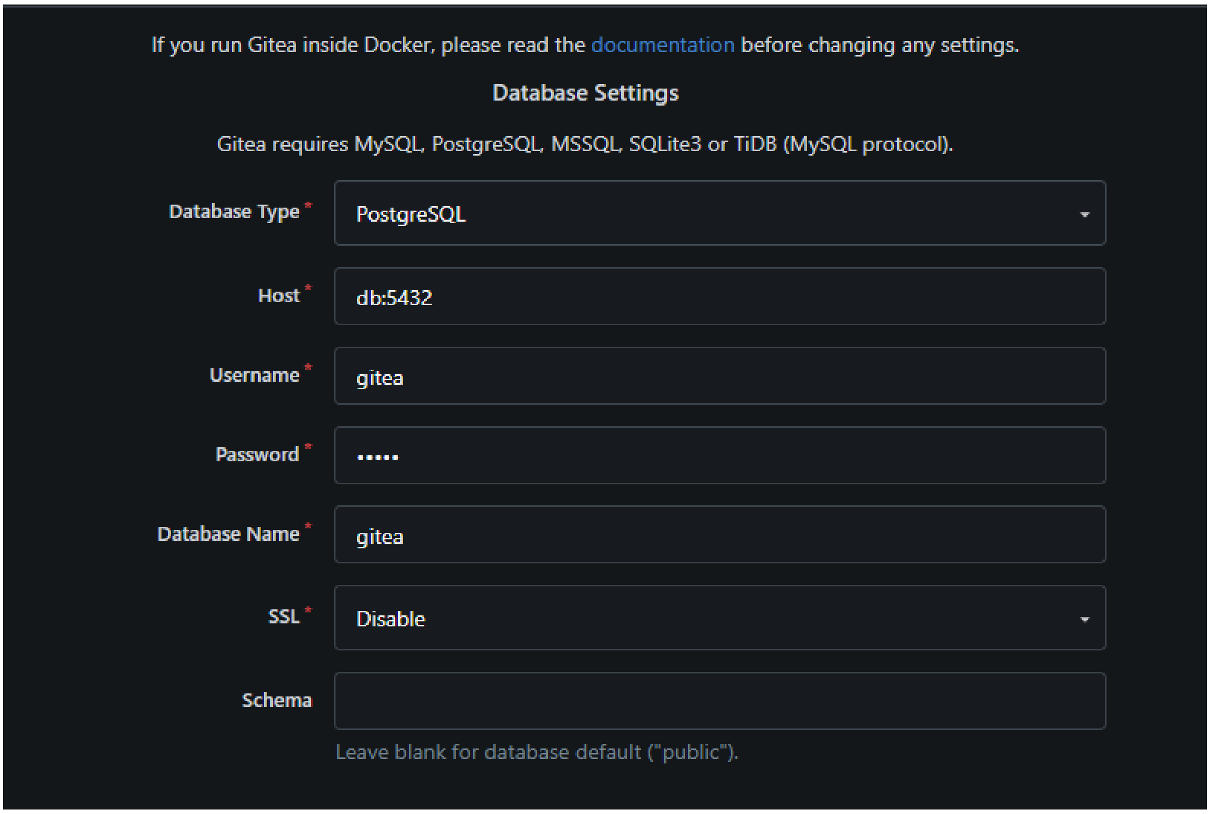
10.Edit the security group Lab8SecurityGroup inbound rules in the EC2 console: add Custom TCP rule port 3000 source 0.0.0.0/0 and save. Capture the inbound rules after saving:

- Save screenshot as: task4_security_group_allow_3000.png — security group inbound rules list showing SSH from My IP and Custom TCP 3000 anywhere.



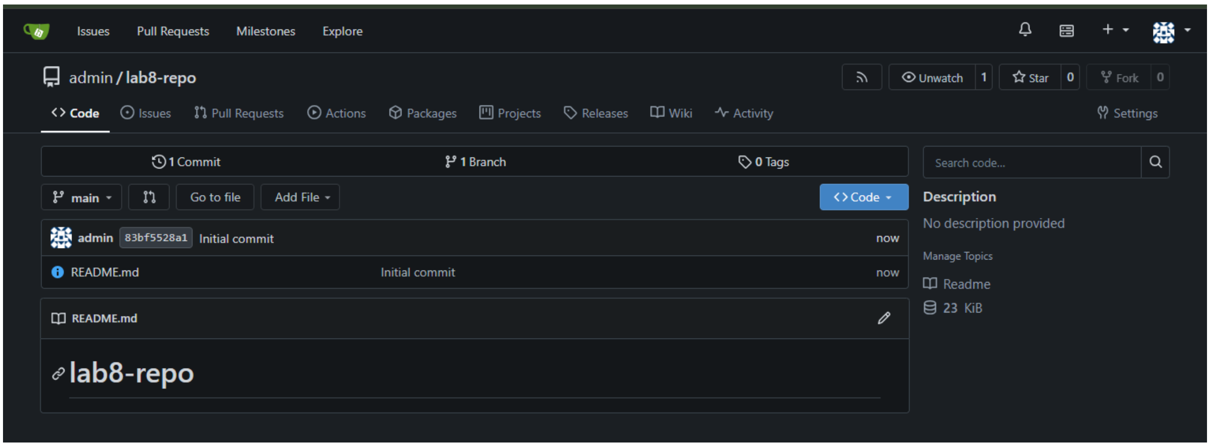
11.From your Windows browser navigate to: http://Public-IP:3000 — capture the Gitea setup/install page:

- Save screenshot as: task4_gitea_install_page.png — Gitea installation page in browser.



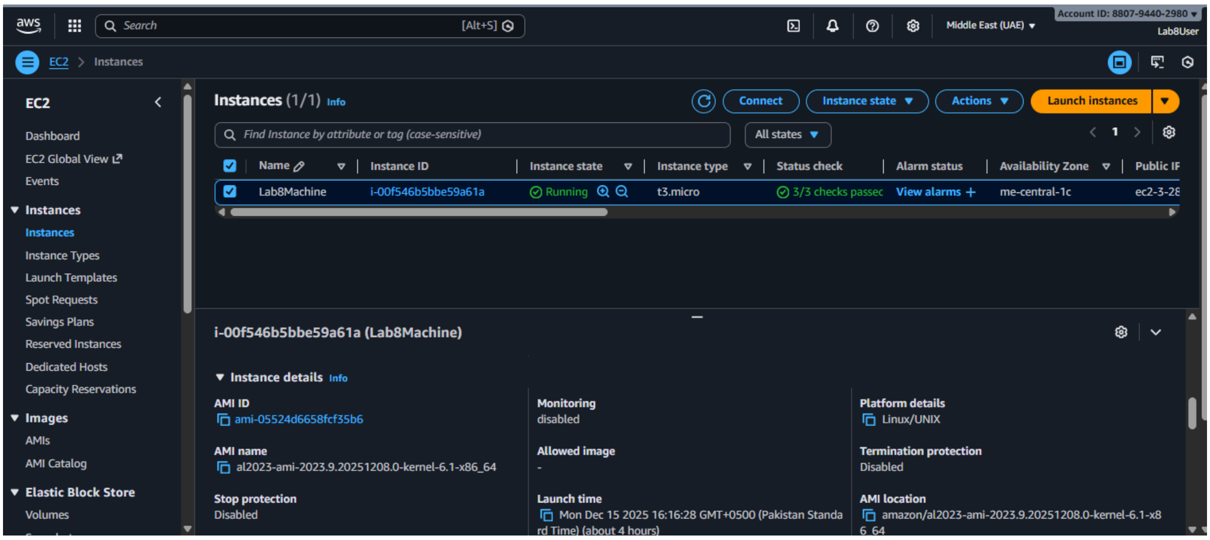
12.Complete initial Gitea setup (create admin user, create a repo) and capture Gitea showing the created repository:

- Save screenshot as: task4_gitea_create_repo.png — Gitea UI showing the created repository.



13.Task 4 summary (combine evidence)

- Save screenshot as: task4_summary.png — single screenshot (or tiled screenshot) showing: EC2 Instances list with Lab8Machine running and public IP, security group inbound rules showing SSH and port 3000, and browser tab open to Gitea UI or repo list.



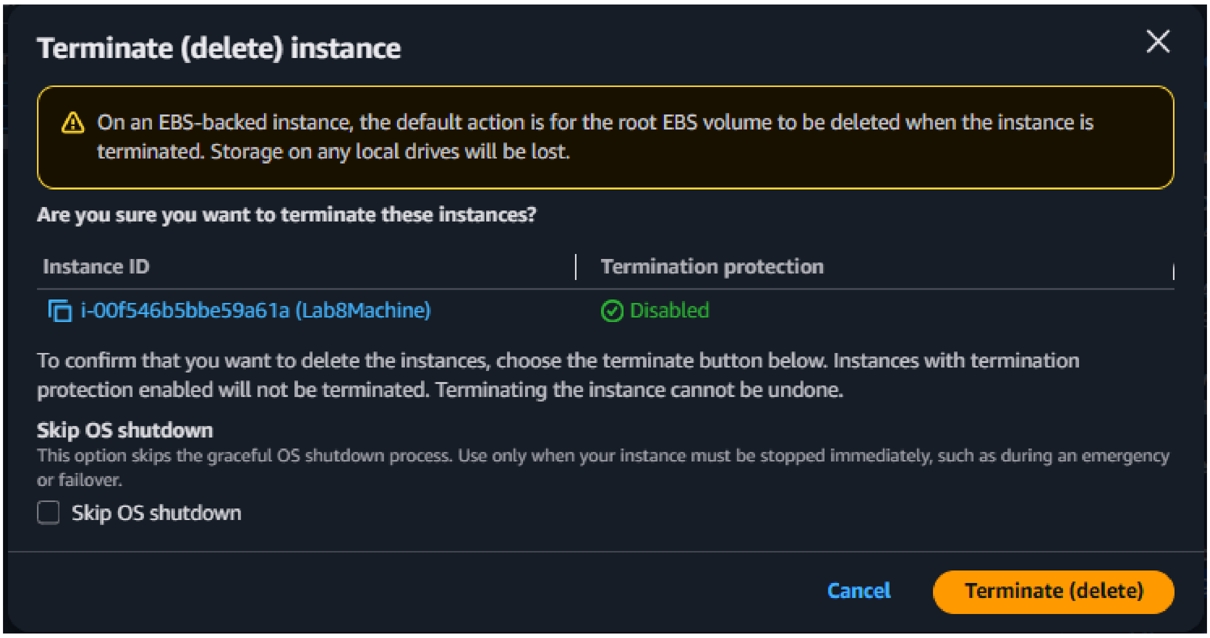
Cleanup — Remove resources to avoid charges

After verification, terminate and delete everything you created. Capture screenshots immediately after each cleanup step.

Cleanup steps and required screenshots:

1. Terminate the EC2 instance Lab8Machine.

- Save screenshot as: cleanup_terminate_instance.png — EC2 terminate instance confirmation.

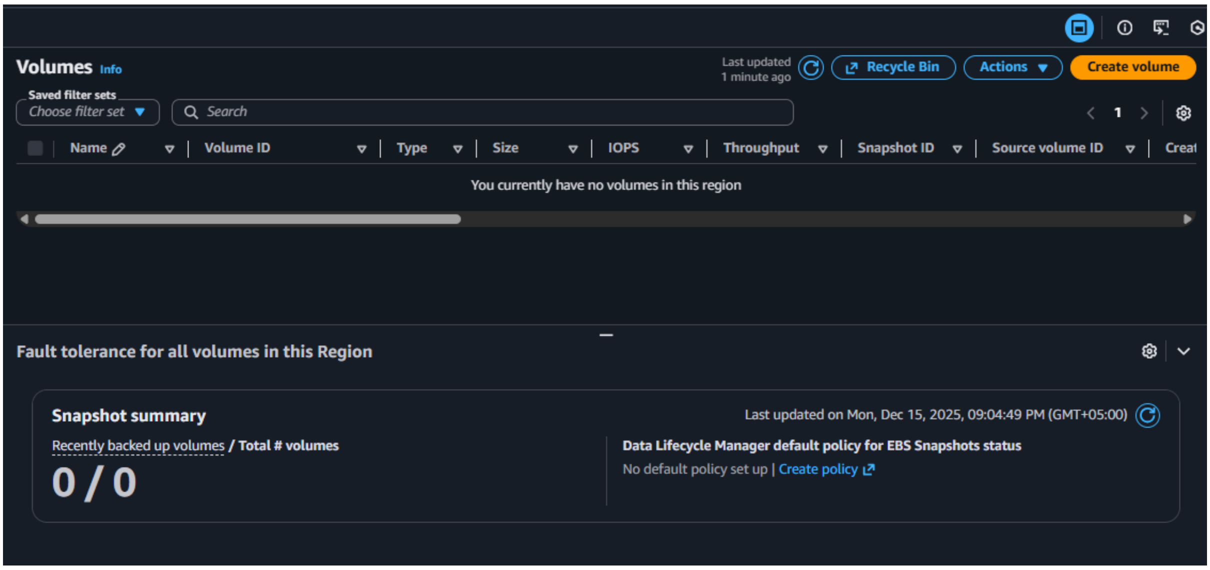


2.Delete associated EBS volumes and snapshots (if any).

- Save screenshot as: cleanup_delete_volumes_snapshots.png — confirmation or list showing volumes/snapshots deleted.

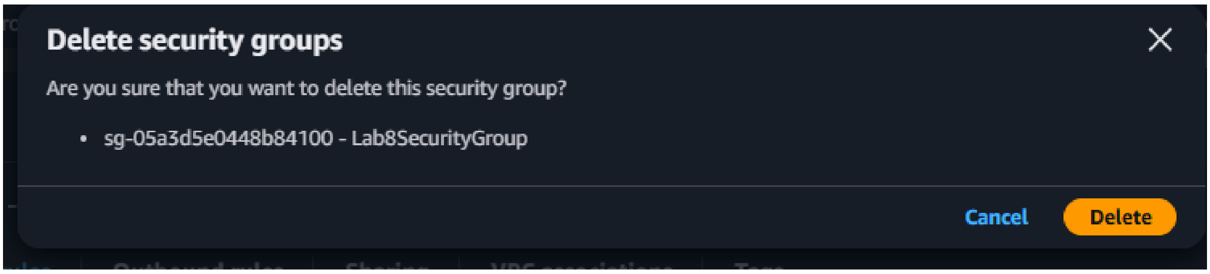
3.Delete associated EBS volumes and snapshots (if any).

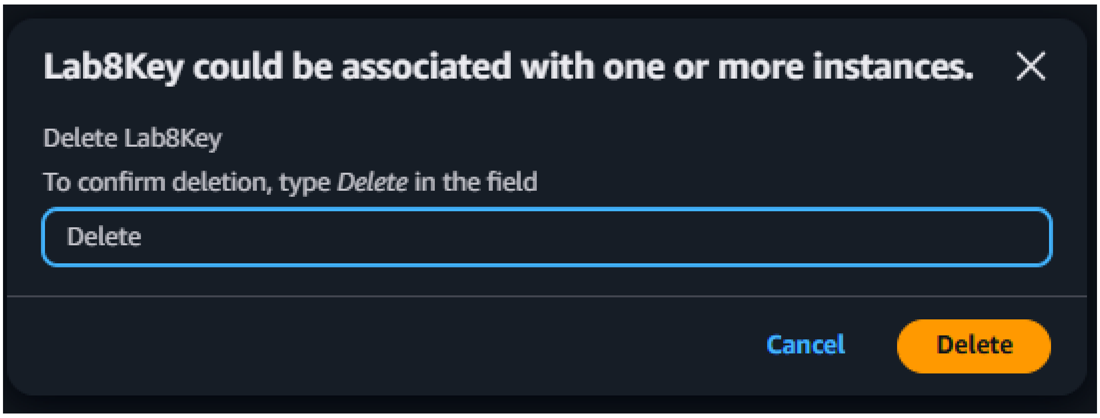
- Save screenshot as: cleanup_delete_volumes_snapshots.png — confirmation or list showing volumes/snapshots deleted.



3.Delete security group Lab8SecurityGroup and key pair Lab8Key from the EC2 console (after instances terminated).

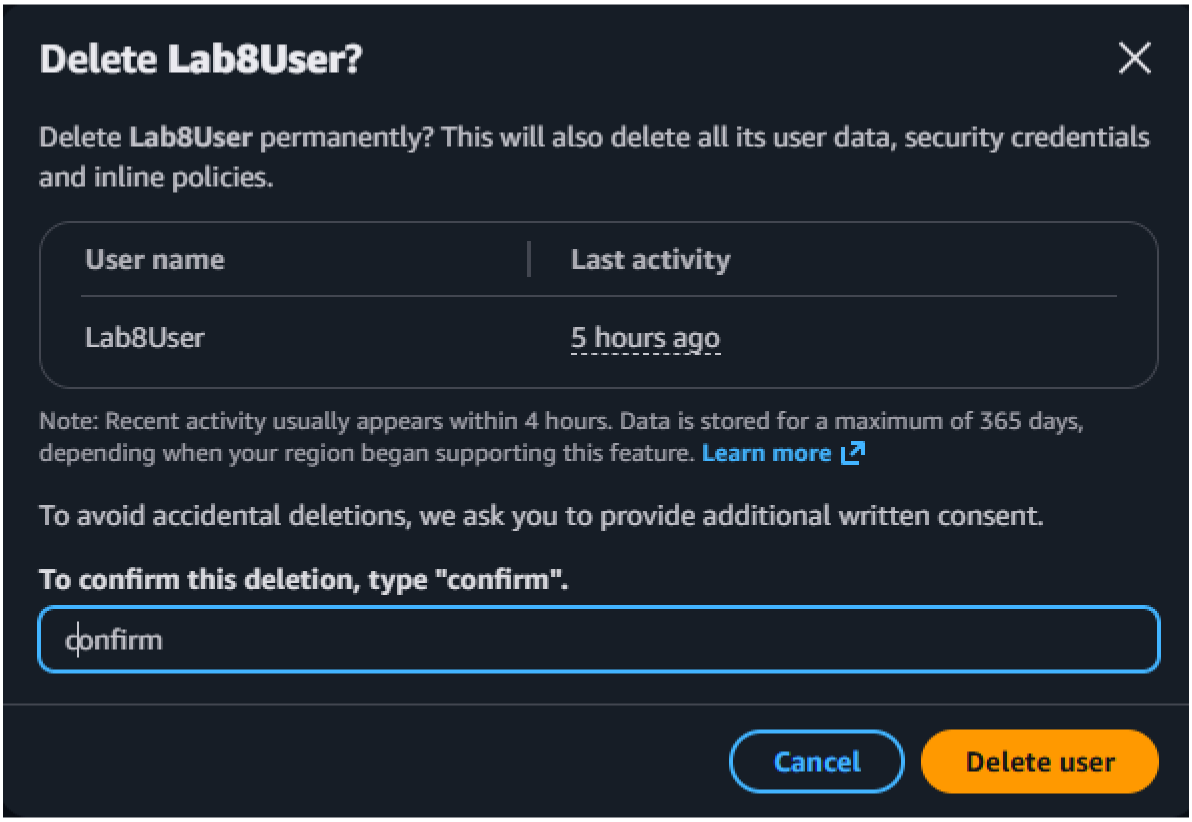
- Save screenshot as: cleanup_delete_security_group_and_keypair.png — deletion confirmation(s) (show key pair list and security group list after deletion).





4.Delete IAM users Lab8User and any access keys.

- Save screenshot as: cleanup_iam_users_deleted.png — IAM Users list showing Lab8User no longer present (or a deletion confirmation).



5.Final cleanup summary (show billing or resource groups with no active resources if possible).

- Save screenshot as: cleanup_summary.png — AWS console Billing/Resource Groups showing no active resources or no recent charges (if available).